Multi-Stage AI Workflow Project Report

To-Do List Web Application

Planning with ChatGPT · Implementation with Claude Code · Verification with GitHub Copilot CLI

Date: Oct 17, 2026

1. Introduction

1.1 Project Overview

This project is a simple To-Do List web application built with Python, Flask, HTML, CSS, Jinja2 and SQLite. Users can add, view, edit, delete, complete and filter tasks, and all tasks are saved in a database so they remain after a restart. The project also demonstrates a multi-stage AI workflow in which three AI tools with different interfaces each handle one stage: ChatGPT (web chat) for planning, Claude Code in VS Code (IDE) for implementation, and GitHub Copilot CLI (terminal) for verification.

1.2 Problem Statement

People often track tasks on paper or in scattered notes, which makes it easy to forget work or lose track of what is done. A second problem is that AI-generated code is often accepted without a clear specification or independent checking. This project addresses both by building a small task manager from written requirements and verifying it with a separate AI tool.

2. AI Workflow Design

2.1 Workflow Overview

The workflow follows a simple principle: plan first, build second, verify third. ChatGPT's output was saved as a Markdown requirements document, which Claude Code used to build the application. Copilot CLI then tested the finished code against the same requirements. Using a different tool for verification means the code is not checked by the tool that wrote it.

2.2 AI Tools and Their Roles

Stage	AI Tool	Interface Type	Input	Output
1. Planning	ChatGPT	Web chat interface	Project idea and planning prompt	Requirements, acceptance criteria, implementation plan, testing checklist
2. Implementation	Claude Code	Agent inside VS Code (IDE)	Planning Markdown files	Flask source code, templates, database setup and tests
3. Verification	GitHub Copilot CLI	Terminal (command line)	Requirements, checklist and source code	Test results and PASS / FAIL / NOT VERIFIED report

2.3 Workflow Diagram

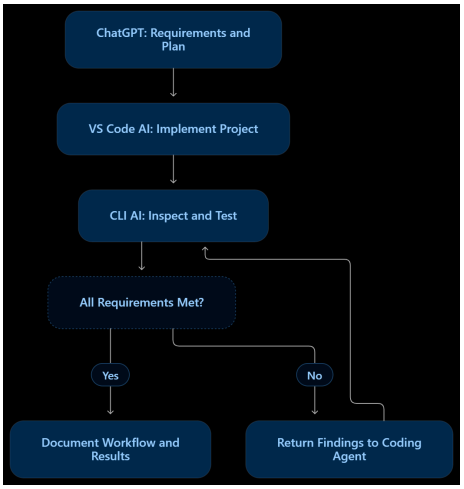


Figure 1: Multi-stage AI workflow: ChatGPT → Claude Code (VS Code) → GitHub Copilot CLI

3. Stage 1 — Requirements and Planning (ChatGPT)

3.1 Purpose of This Stage

ChatGPT acted as a requirements analyst. It was told not to write code, so the planning stayed focused on what to build and how to check it.

3.2 Prompt Used

I want to build a simple To-Do List web application using Python, Flask, HTML, CSS, and SQLite.

Users should be able to add, view, edit, delete, complete, and filter tasks. Tasks must be saved in a database so they remain available after restarting the application.

Act as my software requirements analyst and help me prepare before coding. Provide:

1. **A brief project overview and objectives.**
2. **Functional and non-functional requirements with unique IDs.**
3. **User interface requirements.**
4. **Acceptance criteria for each functional requirement.**
5. **A simple project folder structure.**
6. **A step-by-step implementation plan.**
7. **A testing and verification checklist.**
8. **Definition of Done and required documentation.**

Keep everything simple, practical, and beginner-friendly. Build a traditional Flask web application, not an API. Avoid unnecessary features.

Do not write the application code yet. I only want the project requirements and implementation plan to use as instructions for my coding agent later.

3.3 Planning Output

ChatGPT produced one requirements document containing nine functional requirements (FR-01 to FR-09: add, view, edit, delete, complete, filter, persist, validate input and handle missing tasks), eight non-functional requirements, UI rules, acceptance criteria, the database design, folder structure, a seven-phase implementation plan, a test checklist, a Definition of Done and instructions for the coding agent.

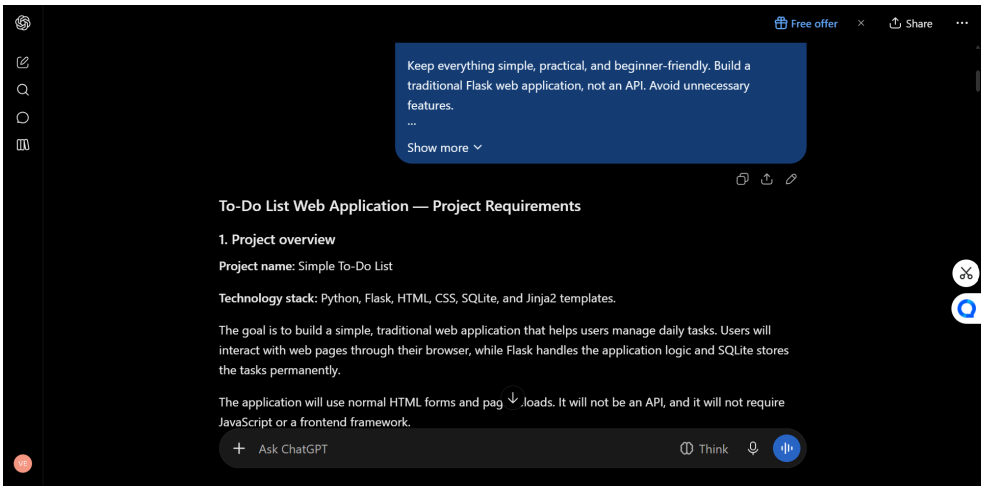


Figure 2: ChatGPT planning prompt and generated requirements

4. Stage 2 — Implementation (Claude Code in VS Code)

4.1 Purpose of This Stage

Claude Code worked inside VS Code, where it could read the requirements, create files and run commands. It was given the requirements document with the prompt below, which came from the coding-agent instructions ChatGPT wrote.

4.2 Prompt Used

Build the Simple To-Do List web application according to the requirements document provided.

Use Python, Flask, SQLite, HTML, CSS, and Jinja2. Build a traditional server-rendered web application, not an API.

Implement the requirements in phases, following the specified folder structure, acceptance criteria, and Definition of Done. Keep the code beginner-friendly and avoid unnecessary features.

Before coding, review the requirements and identify any conflicts or missing details. Then implement one phase at a time, run the relevant tests, and fix any failures before proceeding.

Use parameterized SQL queries, validate inputs, enable CSRF protection, and ensure tasks persist after application restarts. Write automated tests using pytest and a temporary test database.

Do not claim a requirement is complete unless its acceptance criteria have been verified. Finish by updating the README with accurate installation, running, usage, and testing instructions.

At the end, report which requirements are complete, which tests passed, any remaining limitations, and the commands needed to run and test the application.

4.3 Implementation Process

Claude Code followed the seven phases of the plan and wrote 56 automated tests with pytest, each using a temporary database. It fixed one real problem along the way: importing the app in tests created the real tasks.db file, so it switched to an application factory (create_app). Where the requirements were open, it chose an inline confirm() dialog for deletion, limits of 100 characters for titles and 1,000 for descriptions, HTTP 400 for invalid submissions and newest-first ordering.

Claude Code reported all 56 tests passing, also in a clean copy with a fresh virtual environment. It honestly marked FR-04 as only partly verified, because the delete confirmation dialog was never clicked in a real browser.

4.4 Implementation Evidence

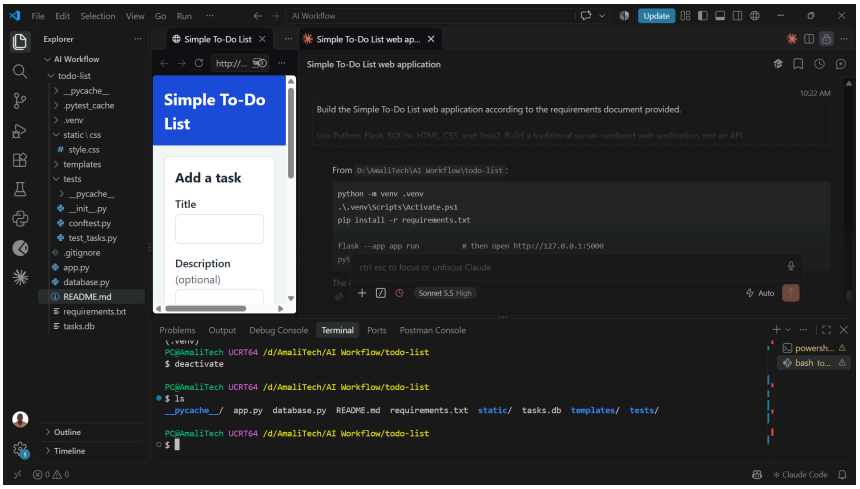


Figure 3: Claude Code working on the project in VS Code

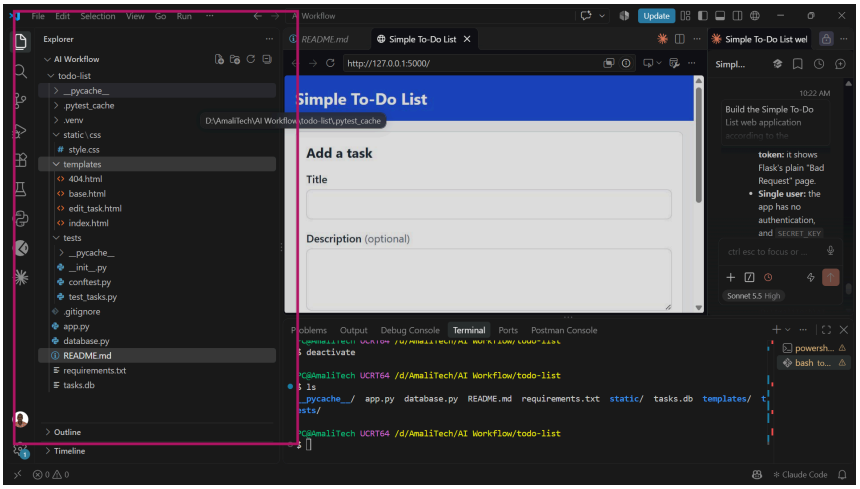


Figure 4: Project folder structure in VS Code

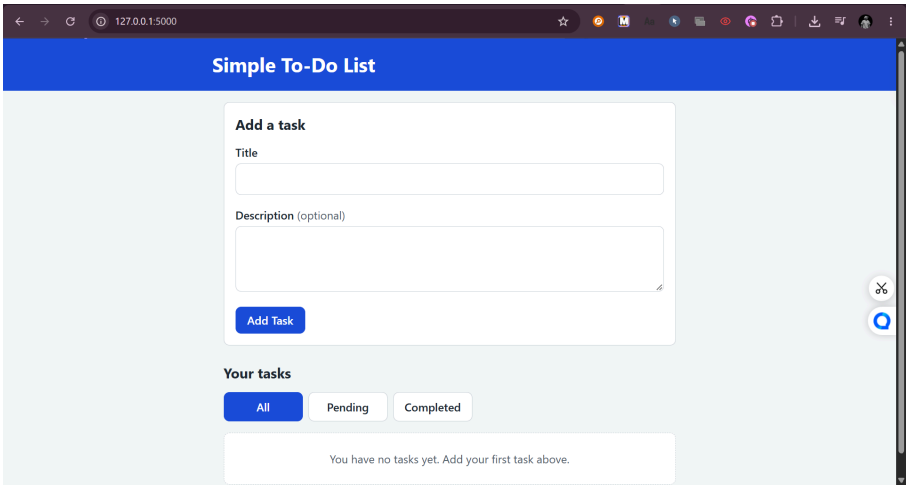


Figure 5: Application home page with the task list

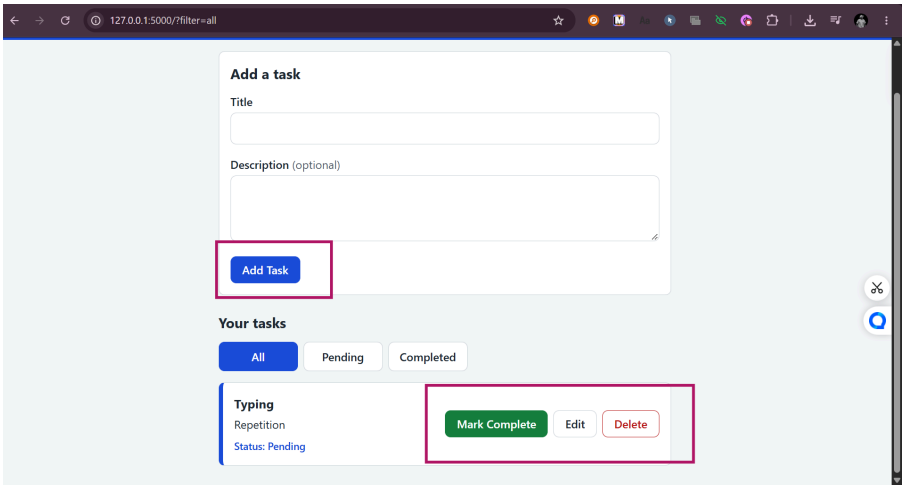


Figure 6: Adding, editing and completing a task

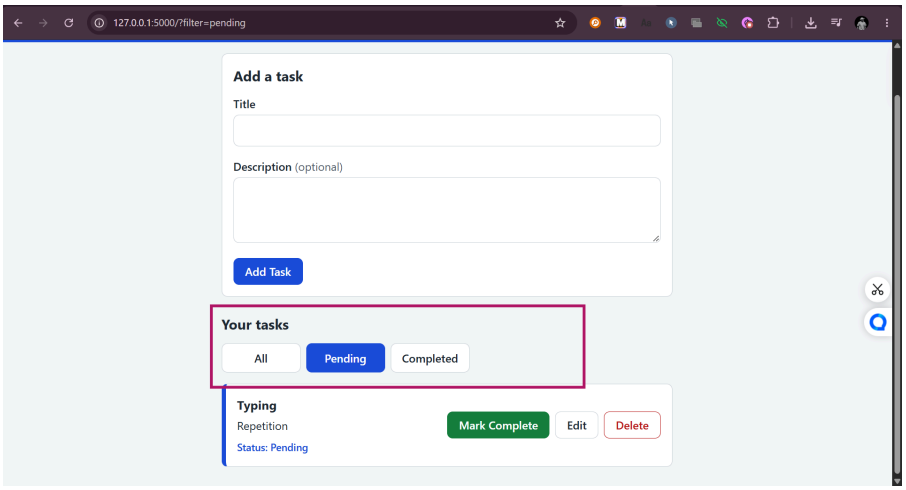


Figure 7: Filtering tasks by status

5. Stage 3 — Verification (GitHub Copilot CLI)

5.1 Purpose of This Stage

Copilot CLI acted as an independent QA engineer in the terminal. It inspected the code, ran the tests, checked each requirement and reported PASS, FAIL or NOT VERIFIED with evidence, without modifying any code.

5.2 Prompt Used

Act as a QA engineer and test my To-Do List web application built with Flask, HTML, CSS, and SQLite.

1. **Inspect the code and project structure.**
2. **Run the existing tests.**
3. **Verify adding, viewing, editing, deleting, completing, and filtering tasks.**
4. **Check whether tasks persist after restarting the application.**
5. **Compare the results against the project requirements.**
6. **Report each requirement as PASS, FAIL, or NOT VERIFIED, with evidence.**
7. **Identify bugs and suggest fixes.**

Do not modify the code yet. Test the actual implementation and report honest results.

5.3 Verification Results

Copilot CLI ran pytest and all 56 tests passed in 1.77 seconds.

ID	Requirement	Result	Evidence
FR-01	Add tasks	PASS	Tasks are created with a title and optional description.
FR-02	View tasks	PASS	Tasks appear on the main page with descriptions and statuses.
FR-03	Edit tasks	PASS	Titles and descriptions are updated successfully.
FR-04	Delete tasks	PASS	Tasks are removed from the application and the database.
FR-04	Delete confirmation	NOT VERIFIED	Confirmation JavaScript is present, but no browser automation was used to interact with the dialog.
FR-05	Complete / return to pending	PASS	Tasks can be completed (with completed styling) and moved back to pending.
FR-06	Filter tasks	PASS	All, Pending and Completed filters work; the filter is kept after actions.
FR-07	Persistence after restart	PASS	Tasks, edits and status survive restarts with the same database.
FR-08	Input validation	PASS	Empty, whitespace-only and oversized values are rejected.
FR-09	Missing-task handling	PASS	Unknown or invalid IDs return a friendly 404 response.

A separate live test with two Flask processes (add, complete, filter, edit, restart, check persistence, delete) also passed. CSRF protection and responsive markup were confirmed

by tests. Copilot CLI's overall result was PASS with no reproducible bugs, matching Claude Code's own report, so no fixes had to be sent back.

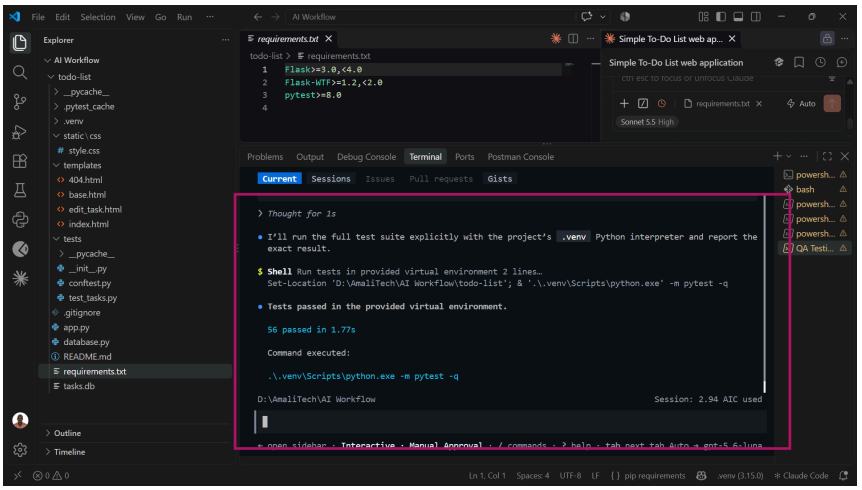


Figure 8: GitHub Copilot CLI running the automated tests

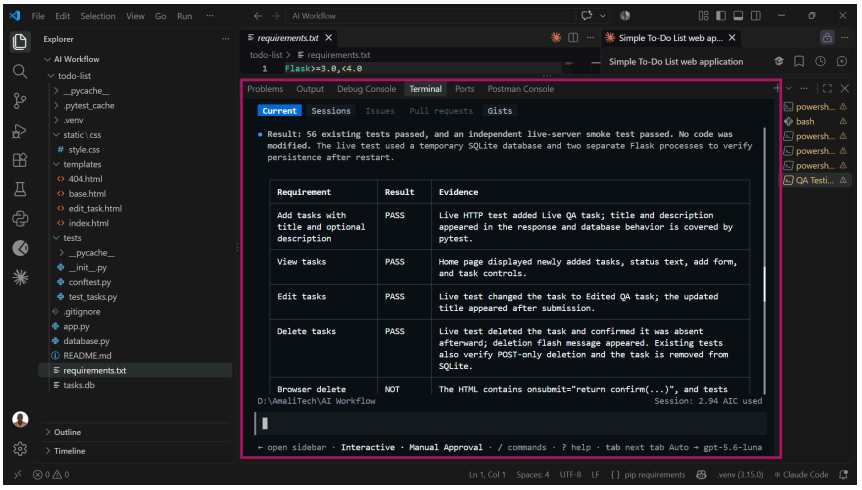


Figure 9: GitHub Copilot CLI requirements verification report

6. Final Results

The final application works locally in the browser and meets all nine functional requirements. Tasks persist in SQLite, forms are CSRF-protected, queries are parameterized and input is validated.

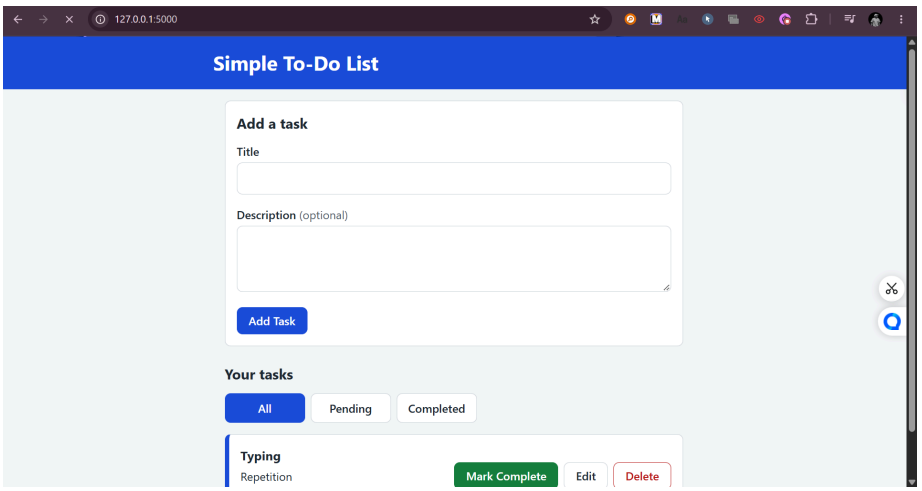


Figure 10: Final To-Do List application

6.1 Known Limitations

- The delete confirmation dialog was not tested by browser automation.
- Testing used only Microsoft Edge and Python 3.15; performance was not measured.
- There are no user accounts, and dependencies are not pinned to exact versions.

7. Evaluation and Lessons Learned

Writing requirements with acceptance criteria first made it clear what "finished" meant, and both AI agents worked from the same document. Each tool was used where its interface was strongest: chat for planning, the IDE for coding and the terminal for testing. The test-database requirement even helped Claude Code catch a hidden bug.

The main limitations were that context had to be passed between tools manually, and that AI agents cannot check everything; neither could click the delete dialog. AI output still needs human review.

8. Conclusion

This project showed that a multi-stage AI workflow can turn a simple idea into a planned, built and verified application. ChatGPT defined the requirements, Claude Code built the app with 56 passing tests, and GitHub Copilot CLI independently confirmed all nine functional requirements. AI tools work best with clear written requirements and when their work is checked rather than trusted automatically.